Building Chronicle with Claude Code

A beginner's guide to designing and building your history app in VS Code, one small, affordable step at a time.

For Bob · Chronicle project · October 2026

Covers: VS Code basics, the ticket workflow, choosing models, keeping costs down, turning ideas into features, and fixing things when they go wrong.

Contents

1. The big idea: why some conversations cost more than others
2. Your toolkit: VS Code, Claude Code and the project files
3. Getting comfortable with VS Code and Claude Code
4. The ticket workflow, step by step
5. Choosing a model
6. Plan mode: thinking before building
7. Writing good requests
8. Checking the work yourself
9. Keeping a conversation lean
10. Designing the app: from idea to ticket
11. Saving your work with Git
12. When things go wrong
13. Cheat sheet

How to read this guide

Read sections 1 to 4 once, slowly. They are the whole system. The rest is reference: come back to it when you need it. Keep the cheat sheet (the last page) open next to VS Code for your first few sessions.

Anything in a dashed box marked **You type** is something you can copy word for word into Claude.

1. The big idea

Claude works in **tokens**, small pieces of text (roughly ¾ of a word each). Everything Claude reads or writes is counted in tokens: your messages, its replies, every file it opens, every test result it sees.

Here is the part that matters most: **each time you send a message, Claude re-reads the whole conversation so far.** It has no memory between messages other than the conversation itself.

> **AN ANALOGY**
>
> Imagine a research assistant who, before answering each new question, re-reads the entire notebook of everything you've discussed today. On page 2 that's quick. On page 200 it takes ages, even if your new question is tiny. That's what a long Claude conversation is like.

So costs grow in two ways:

- **Long conversations.** Message 50 costs far more than message 5, because Claude carries everything from messages 1 to 49.
- **Noise.** Giant test logs, whole files read "just in case", a dozen screenshots: all of it stays in the conversation and gets re-read every turn.

The efficiency system you now have is built on three habits:

1. **Small, well-defined jobs** ("tickets"), one per conversation.
2. **The right model** for each job: cheap for simple work, powerful for design.
3. **Quiet tools**, so the conversation fills with decisions, not noise.

None of this lowers quality. A short, focused conversation usually produces *better* work, because Claude isn't distracted by stale information.

> **THE ONE RULE TO REMEMBER**
>
> One ticket, one conversation. When it's done and committed, start a fresh conversation for the next thing.

2. Your toolkit

The programs

Tool	What it is
VS Code	The editor where your project's files live. You can read code here, but you mostly won't need to.
Claude Code	Claude inside VS Code. It can read and edit files, run commands and test the app, with your permission.
Terminal	A text window inside VS Code for running commands such as `npm run dev`. Open it with Ctrl + ` (the key above Tab).
Git	Saves snapshots ("commits") of the project so nothing is ever lost. Claude makes the commits for you.

The project files that run the system

These were set up for you. You don't need to edit most of them, but it helps to know what each does.

File	What it does	Do you edit it?
ROADMAP.md	The to-do list. The **Next tickets** section at the top holds ready-to-run jobs.	Yes, to add ideas.
CLAUDE.md	The project briefing Claude reads at the start of every conversation: what Chronicle is, the rules, the commands. Kept short on purpose, because it's read every single turn.	Rarely.
src/ui/CLAUDE.md src/model/CLAUDE.md e2e/CLAUDE.md	Specialist notes that only load when Claude works in that folder (screens, data, tests). They keep the main briefing small.	No.
docs/decisions.md	Why things were built the way they were, so you don't re-argue settled questions.	Claude adds to it.
docs/design.md	Colors, fonts and wording style.	When you change taste.
.claude/settings.json	Permissions: what Claude may run without asking, and huge files it must never read.	No.
Claude's memory	Notes Claude keeps about your preferences (for example, "one ticket per session") so you don't repeat yourself.	Ask Claude to "remember …".

3. Getting comfortable with VS Code and Claude Code

Opening the project

1. Open VS Code.
2. Choose File → Open Folder… and pick `Documents/code/chronicle`.
3. The file list appears on the left. `ROADMAP.md` is near the bottom.

Opening Claude

- Click the Claude (spark) icon at the top right of the editor, or press `Cmd`+`Esc`.
- A Claude panel opens with a message box at the bottom. Type there and press `Enter` to send.
- To start a **new conversation**, use the new-conversation button at the top of the Claude panel, or type `/clear`.

Slash commands

Messages that start with `/` are commands for Claude Code itself, not requests to Claude. Type `/` to see a list. The ones you'll use:

Command	What it does
`/clear`	Starts over with an empty conversation.
`/model`	Switches model, e.g. `/model sonnet`.
`/context`	Shows how full the conversation is and what's filling it.
`/compact`	Squeezes the conversation into a summary to make room.
`/cost`	Shows what this session has used. (On a subscription plan, `/usage` shows your limits.)

If a command doesn't appear in the panel, open VS Code's terminal and type `claude`: the terminal version has every command.

Pointing at files

Type `@` in the message box and start typing a file name, e.g. `@ROADMAP.md`. This tells Claude exactly which file you mean, which is cheaper than letting it search. If you highlight lines in the editor, Claude can see your selection too.

Permission prompts

Before Claude edits a file or runs a command it hasn't been pre-approved for, it asks. You'll see what it wants to do, with options to allow or deny.

- **Allow** edits to project files and the usual commands (`npm run …`, `git …`).
- **Pause and ask** if it wants to delete things outside the project, install unfamiliar software, or push to the internet. Just reply “why do you need that?”

Seeing changes

When Claude edits a file, VS Code can show a **diff**: removed lines in red, added lines in green. You don’t need to understand the code, but skimming a diff tells you how big a change was.

TRY IT NOW (2 MINUTES)

Open Claude, start a new conversation, then:

> **You type:** /model haiku

> **You type:** In one paragraph, what are the next tickets in @ROADMAP.md?

That’s a full, cheap round trip. You’ve used a slash command, a cheap model and an @-mention.

4. The ticket workflow, step by step

A **ticket** is one small job with a clear finish line. Each one in `ROADMAP.md` looks like this:

```
1. **Place picker** · S · sonnet. Set an entry's places by hand …
   Files: src/ui/panels/ItemPanels.tsx, src/media/wikipedia.ts
   Done when: add, edit and remove places in the editor; an e2e test
   adds a place and sees the pin label on the map.
```

The labels mean: **S/M** is size (S is one sitting, M a longer one). **sonnet** is the model to use. **Files** is where Claude should start, which saves it from searching. **Done when** is how you both know it's finished.

1. **Start fresh.** Open a new conversation (button, or `/clear`). Never carry a finished ticket's conversation into the next one.

2. **Set the model the ticket names.**

   > **You type:** /model sonnet

3. **For size M only, turn on plan mode** (see section 6): press Shift + Tab until the mode shows "plan", or pick it from the mode menu under the message box.

4. **Ask for the ticket.**

   > **You type:** Do ticket 1 from ROADMAP.

   Add anything you care about: *"…and keep the place list short on phones."*

5. **Answer questions, approve the plan.** Claude may ask one or two questions or (in plan mode) show a plan. Read it like a builder's quote: does it do what you wanted? Say "yes" or change it.

6. **Let it work.** Claude edits files, writes tests, runs `npm run check:quiet` and, for screen changes, takes one screenshot sheet. Approve permission prompts as they come.

7. **Look at it yourself** (section 8): run the app and try the feature for a minute.

8. **Commit and close.**

   > **You type:** Looks good. Tick the ticket and commit.

   Then start a fresh conversation for whatever's next.

WHY THIS WORKS

Each conversation stays small, so every message stays cheap. The ticket carries the context Claude would otherwise spend tokens discovering. And the commit means the next conversation can start from scratch without losing anything.

5. Choosing a model

Claude comes in sizes. Bigger models think harder and cost more. Use the smallest one that can do the job well.

Model	Cost	Use it for	Chronicle examples
Haiku `/model haiku`	Lowest	Small, clear, mechanical jobs. Questions about the project.	Reword a button. Update docs. The backup reminder. “What does ticket 3 involve?”
Sonnet `/model sonnet`	Middle	Normal features where the approach is clear.	Place picker, search, tech tree.
Opusplan `/model opusplan`	Middle to high	Features that need design thinking. Uses Opus while planning, Sonnet while building.	Meanwhile, Review mode.
Opus `/model opus`	Highest	Big decisions and stubborn problems.	“Should sync use GitHub or a server?” A bug that two attempts didn’t fix.

WHEN UNSURE

Use `opusplan`. It’s the best all-rounder: good thinking where it matters, cheaper hands for the typing.

SIGNS YOU PICKED TOO SMALL A MODEL

- It misunderstands the same thing twice.
- A fix breaks something else, then the next fix breaks something else again.

Stop, then `/model opus` and say: “Step back. Here’s what we tried and what went wrong. What’s the real problem?”

6. Plan mode: thinking before building

In plan mode, Claude reads and thinks but **changes nothing**. It ends with a written plan for you to approve. Nothing happens until you say yes.

When to use it

- **Yes:** M-sized tickets, anything that changes how data is saved, and new screens or views.
- **No:** small tickets, wording changes, bug fixes with an obvious cause.

How to use it

1. Turn it on: Shift+Tab (cycles modes) or the mode menu under the message box.
2. Ask for the ticket as usual.
3. Claude may ask you questions with clickable options. Pick one, or choose “Other” and type your own.
4. Read the plan. Check three things: *does it do what I want*, *is anything missing*, *is anything there I didn’t ask for*.
5. Approve, or reply with changes: “Drop the settings page; just put the toggle in the Layers menu.”

> **WHY PLAN MODE SAVES TOKENS**
>
> Changing a plan costs a sentence. Changing finished code costs a rebuild. Five minutes reading a plan can save an hour of corrections.

7. Writing good requests

A good request answers three questions: **what**, **where**, and **how will we know it's done**. The more of that you supply, the less Claude spends working it out.

COSTS MORE

"The map feels off, can you look at it?"

Claude has to explore the map code, guess what "off" means, and maybe ask anyway.

COSTS LESS

"On the map at 300 BCE, territory names overlap pin names. Pins should win. Done when both are readable in the map screenshot."

COSTS MORE

Three separate messages: "make Play bigger" … "also the year font" … "oh and the speed menu".

COSTS LESS

One message: "In the year bar: bigger Play button, larger year text, and default speed 10 years/s."

A template you can reuse

```
Goal: <what you want, in plain words>
Where: <screen or @file, if you know it>
Done when: <what you'll see or be able to do>
Keep in mind: <anything important to you>
```

Useful phrases

- **"Ask me before you start if anything is unclear."** Cheap questions beat expensive guesses.
- **"Just tell me, don't change anything."** For questions about how something works.
- **"Smallest change that works."** Stops gold-plating.
- **"Turn this idea into a ticket."** See section 10.

AVOID

- Pasting long logs or whole files. Paste the one or two lines that matter; Claude can rerun things itself.
- "Review everything thoroughly." Name what worries you instead.
- "While you're at it…" halfway through a ticket. Write it down as a new ticket.

8. Checking the work yourself

Claude runs automated tests, but you're the one who knows whether a feature feels right for learning and exploring.

Run the app

1. Open the terminal: Ctrl+`.
2. Type `npm run dev` and press Enter.
3. Open the address it prints (usually `http://localhost:5173`) in your browser.
4. Try the new feature for a minute. Stop the app later with Ctrl+C in the terminal.

Your browser keeps your own timeline. To try things with sample data, use More → Import a file… and pick `tests/fixtures/sample-history.json`. Export a backup of your real timeline first.

What to look for

- **Learning:** does it help me remember or connect something?
- **Notes:** is my writing easy to find from here?
- **Curiosity:** does it make me want to look something up?
- The basics: readable on a phone-sized window, readable in dark mode, sensible wording.

The checks Claude runs

Command	What it checks	Output
`npm run check:quiet`	Code types, unit tests, the build, and browser tests of the main flows.	A few lines when everything passes; only failures otherwise.
`npm run check`	The same, in full detail.	Only for digging into a failure.
`npm run shots -- --only map --sheet`	Takes the screenshots whose names match, tiled into one image.	`screenshots/sheet.png`

Screenshot names include `desktop-timeline`, `desktop-entry`, `desktop-map`, `desktop-split`, `desktop-dark`, `phone-timeline` and `phone-map`. A fragment like `map` matches every name containing it.

9. Keeping a conversation lean

Even with one ticket per conversation, some tickets run long. Here's how to keep them in check.

Situation	Do this
The ticket is done	Commit, then `/clear` or open a new conversation.
Mid-ticket, the conversation feels long or slow	`/context` to see how full it is. If it's over about half, compact (next row).
You need to keep going but trim the history	`/compact keep the plan, the files changed and the failing test`
You went down a dead end	Ask: "Summarize what we learned in three bullets." Copy them, `/clear`, paste them into the fresh conversation.
Curious what a session used	`/cost` (or `/usage` on a subscription).

> **COMPACT VS CLEAR**
>
> **/compact** is like replacing a long meeting transcript with good minutes: you keep going, with less to carry. **/clear** is like starting a new meeting: cheapest, but you only keep what's in the files, the roadmap and the commits. That's why tickets end with a commit.

Helpers working in the background

- **Folder notes** load only when Claude works in that part of the project.
- **Blocked files.** Huge files (map data, build output, the package lockfile, minified libraries) can't be read. They're never useful to read and would waste thousands of tokens.
- **A silent type checker** runs after every edit and only speaks up, with the first error, when something breaks.
- **Memory** holds your preferences across conversations.

10. Designing the app: from idea to ticket

Good design here means every feature serves at least one of Chronicle's three purposes, and is small enough to build in one sitting.

The three questions

Purpose	Ask	Example features
Learning	Will this help me remember, understand or test myself?	Review mode, study sheets, "Meanwhile".
Notes	Will this make my writing easier to capture, link or find?	Search, `[[links]]` in notes, tags.
Curiosity and fun	Will this make me want to explore or look something up?	The map's Play button, tech tree, connection graph.

If an idea hits none of these, it can probably wait.

Step 1: capture the idea (free)

Open `ROADMAP.md` and jot it at the bottom of "Next tickets" in one sentence. No need to ask Claude yet.

```
- Idea: show books I've read as their own lane on the timeline.
```

Step 2: shape it into a ticket (cheap)

Start a fresh conversation on Haiku or Sonnet:

> **You type:** Turn the "books I've read" idea in @ROADMAP.md into a ticket like the others: size, model, files, done-when. If it's bigger than M, split it. Don't build anything.

You'll get something like:

```
7. **Reading lane** · M · opusplan. Sources with a "read on" date
   appear in their own lane on the timeline.
   Files: src/model/types.ts (schema bump), src/ui/TimelineView.tsx,
   src/ui/panels/SourcePanels.tsx
   Done when: a source with a read date shows in the lane and opens
   on click; old backups still import.
```

Step 3: order the list

Put first whatever gives the most learning, note-finding or fun for the least size. Quick wins (S) that unlock other things, like the place picker, go early.

Step 4: build it with the ticket workflow

Section 4, from the top.

Sizing rules of thumb

- **S:** one screen, no change to how data is saved.
- **M:** a new panel or view, or a change to saved data.
- **Bigger than M:** split it. “Review mode” could become (a) ordering quiz, then (b) spaced repetition.

Exploring without building

Sometimes you want to think out loud about direction (“what would make the map more like Civ?”). That’s fine. Do it in its own conversation on `opusplan`, ask for a short list of ideas, and save the good ones as one-line ideas in the roadmap. Then `/clear`. Brainstorms and builds don’t mix well in one conversation.

> **YOUR APP’S HOUSE RULES (CLAUDE FOLLOWS THESE AUTOMATICALLY)**
>
> - The app ships empty: you add the history yourself.
> - Real pictures only (no AI images), always with credit.
> - Everything works offline. Internet features say so when you’re offline.
> - Old backups always import, even after changes to how data is saved.

11. Saving your work with Git

Git keeps snapshots of your project called **commits**. Each ticket ends with one, so you can always go back.

You want to…	Say to Claude
Save the finished ticket	“Commit this.”
See recent history	“Show the last 5 commits in one line each.”
Undo the last ticket	“Revert the last commit.” (This adds an undo commit; nothing is lost.)
Throw away unfinished, uncommitted changes	“Discard my uncommitted changes.” Claude will confirm first.
Try something risky	“Make a branch called experiment-x first.”

Right now your work is on a branch called `wikipedia-and-maps`. When you’re happy with it, ask: “Merge wikipedia-and-maps into main.”

> **YOUR DATA IS SEPARATE**
>
> Your actual timeline (entries, notes, sources) lives in your browser, not in Git. Back it up with More → Export a backup now and then, especially before importing test files.

12. When things go wrong

Problem	What to do
A check fails and Claude's fixes keep missing	After two attempts: `/model opus`, then "Step back: what is the actual cause?"
The conversation has wandered off	"Summarize where we are in three bullets," then `/clear` and paste the bullets.
Claude changed things you didn't ask for	"Undo the changes to X; keep only Y." Next time use plan mode.
The app won't start (`npm run dev` errors)	Paste the first error line and say "npm run dev fails with this."
The map is blank	Refresh the page. If it's still blank: "The map is blank in the browser," plus what you see.
A permission prompt looks odd	Deny it and ask "why do you need that?"
Claude seems to have forgotten the setup	"Read @CLAUDE.md and follow it."
You hit a usage limit	Wait for it to reset, or switch to `/model haiku` for small jobs. Long conversations use limits fastest.

Frequently asked

Can I just chat about history with Claude here? You can, but every message also carries the project briefing. For pure history questions, a regular Claude chat is cheaper. Bring what you learn back as entries.

Do I need to understand the code? No. Understand the tickets, the plans and the app itself. Skimming diffs helps you sense change size, nothing more.

Should I ever edit code myself? You can, but tell Claude afterwards ("I changed X in @file") so it doesn't undo it.

13. Cheat sheet

Every ticket	
1. Fresh start	New conversation, or `/clear`
2. Model	`/model` haiku · sonnet · opusplan · opus (as the ticket says)
3. Plan mode (M tickets)	Shift + Tab
4. Ask	“Do ticket N from ROADMAP.”
5. Try it	Terminal: `npm run dev` → localhost:5173
6. Finish	“Tick the ticket and commit.” Then a fresh conversation.

Commands	
`/clear`	Empty conversation
`/context`	How full is it?
`/compact keep …`	Summarize to make room
`/cost` or `/usage`	What’s been used
`@file`	Point at a file exactly
Ctrl + `` ` ``	Open the terminal
Cmd + **Esc**	Open Claude

Good request	
Goal · Where · Done when	“On the map, pins should sit above territory names. Done when both are readable at 300 BCE.”
Batch it	Several small changes in one message.
New idea mid-ticket?	Write it in ROADMAP. Don’t build it now.

Feature check	
Learning?	Helps me remember or understand
Notes?	Helps me capture or find my writing
Curiosity?	Makes me want to explore